

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING

COURSE CODE: CSA43

COURSE OUTCOME COVERED

CO-3: Implement dynamic web applications by utilizing DOM-based client-side scripting and employing Java Servlets for server-side request processing and response handling. (L3)

Assessment Tool : Technical Presentation

Weightage: 20%

QUESTIONS: Total Marks: 20 1. Servlet Architecture & Workflow

Question:

Present the **architecture of Java Servlets** and explain the complete **request–response cycle** with a neat diagram.

Focus Areas:

- Client–Server interaction
- Role of Web Container
- Request and Response objects

2. Servlet Life Cycle Analysis

Question:

Explain the **Servlet Life Cycle** with a sequence diagram and analyze the role of `init()`, `service()`, and `destroy()` methods.

Focus Areas:

- Life cycle phases
- Execution flow
- Performance implications

3. GET vs POST Method Evaluation

Question:

Compare **GET and POST methods** used in form submission. Present scenarios where each method is appropriate.

Focus Areas:

- Data transmission
- Security aspects
- URL limitations

4. Session Tracking Techniques

Question:

Present different **session tracking mechanisms** (HttpSession, Cookies, URL rewriting). Evaluate their advantages and limitations.

Focus Areas:

- Session persistence
- Security concerns
- Use-case comparison

5. Cookies vs Sessions – Critical Analysis

Question:

Analyze and compare **Cookies and Sessions** in web applications. Present real-time use cases. **Focus Areas:**

- Storage location
- Security risks
- Performance considerations

Code of Conduct:

I K Phani Sridhar Yogi 192311091 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

K-P-Srihar

RUBRICS FOR CALCULATION:

Technical Presentation					
Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Technical Content Accuracy	30	Highly accurate, in-depth, and insightful technical explanation	Technically correct content with adequate depth and clarity	Basic concepts presented with limited accuracy and depth	Content contains major technical errors; concepts poorly understood
Problem Identification	25	Problem rigorously analyzed using appropriate and advanced methods	Problem clearly defined with a logical engineering approach	Problem identified but analysis or methodology is weak	Problem statement unclear or incorrect; no logical approach
Use of Tools	20	Advanced tools, well-analyzed data, and highly effective visuals	Appropriate tools and data used effectively with clear visuals	Limited use of tools and basic data interpretation	Minimal or incorrect use of tools and data; poor visuals
Communication	15	Highly engaging, confident, and audience focused delivery	Clear, organized, and professional communication	Understandable but lacks clarity, structure, or confidence	Poor organization, unclear speech, ineffective slides
Professionalism	10	Exemplary professionalism, leadership, and ethical awareness	Professional conduct with effective team collaboration	Basic professionalism; uneven team participation	Lacks professionalism; minimal contribution or ethical awareness

Question 1: Servlet Architecture & Workflow

Introduction

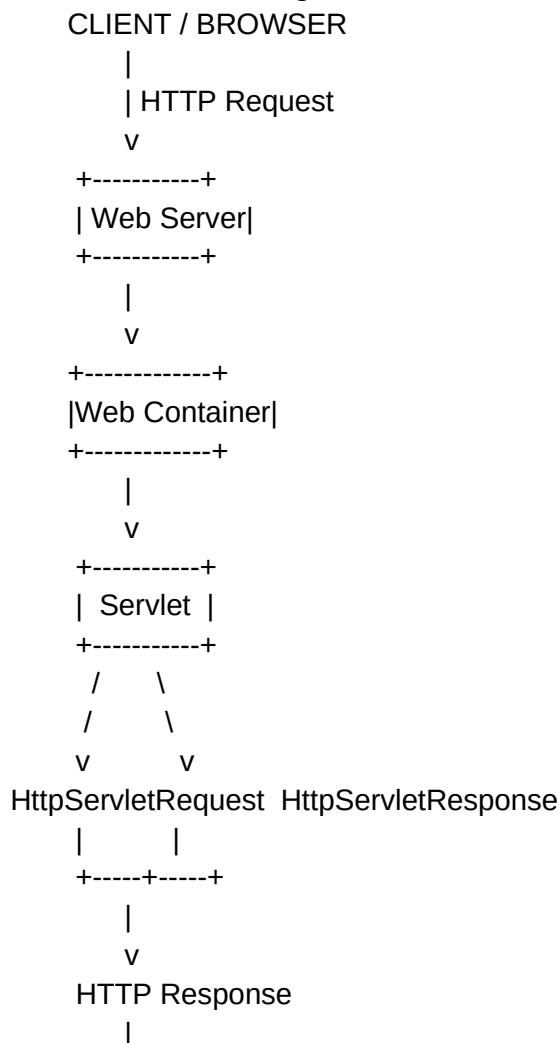
A Java Servlet is a server-side Java program used to handle client requests and generate dynamic responses. Servlets run inside a **Web Container**, which manages the servlet's execution, lifecycle, requests, and responses.

Servlet Architecture

The main components of Servlet architecture are:

1. **Client** – Sends a request to the web server, usually through a web browser.
2. **Web Server** – Receives the client's request and forwards it to the appropriate web application.
3. **Web Container** – Manages servlet execution and provides the runtime environment for servlets.
4. **Servlet** – Processes the client request and generates a response.
5. **Request Object** – Contains information sent by the client, such as parameters and request details.
6. **Response Object** – Used by the servlet to send the generated response back to the client.

Servlet Architecture Diagram



v

CLIENT / BROWSER

Complete Request–Response Cycle

Step 1: Client Request

The client, usually a web browser, sends an HTTP request to the web server.

For example:

http://example.com/login

The request may contain form data, URL parameters, headers, and other information.

Step 2: Web Server Receives Request

The web server receives the HTTP request and identifies the requested web resource.

If the request is mapped to a servlet, it is passed to the Web Container.

Step 3: Web Container Processes Request

The Web Container manages the servlet and creates the required request and response objects.

It provides:

- HttpServletRequest
- HttpServletResponse

The request object contains information from the client, while the response object is used to construct the response.

Step 4: Servlet Processes Request

The servlet receives the request and processes the required business logic.

For example, it may:

- Read form data.
- Validate user input.
- Access a database.
- Perform calculations.
- Generate dynamic content.

Step 5: Servlet Generates Response

The servlet uses the HttpServletResponse object to create the response.

The response may contain:

- HTML
- Text
- JSON
- Other HTTP response data

Step 6: Response Sent to Client

The Web Container sends the generated response through the web server back to the client.

The browser then displays the result to the user.

Request–Response Flow

Client

|

| 1. HTTP Request

v

Web Server

|

| 2. Forward Request



Role of Web Container

The Web Container is a major component of the Servlet architecture. Its responsibilities include:

1. Managing servlet lifecycle.
2. Creating and managing servlet instances.
3. Receiving and forwarding client requests.
4. Creating request and response objects.
5. Managing multiple client requests.
6. Providing security and configuration support.
7. Managing sessions and other web application resources.

Request and Response Objects

HttpServletRequest

The HttpServletRequest object provides information about the incoming request.

It can be used to obtain:

- Form parameters.
- Request headers.
- HTTP method.
- URL information.
- Client-related information.

Example:

```
String name = request.getParameter("name");
```

HttpServletResponse

The HttpServletResponse object is used to send information back to the client.

It can be used to:

- Set response headers.
- Set content type.
- Send output.

- Redirect the client.

Example:

```
response.setContentType("text/html");
```

Advantages of Servlet Architecture

1. Provides server-side processing using Java.
2. Supports dynamic web applications.
3. Provides efficient request handling.
4. Web Container manages the servlet lifecycle.
5. Supports session management.
6. Provides a structured request-response mechanism.

Conclusion

Servlet architecture provides a structured mechanism for developing dynamic web applications. The client sends an HTTP request, which is processed by the web server and Web Container. The servlet processes the request using `HttpServletRequest` and generates a response using `HttpServletResponse`. The response is then returned to the client. Thus, the Web Container plays a central role in managing servlet execution and request-response processing.

Question 2: Servlet Life Cycle Analysis

Introduction

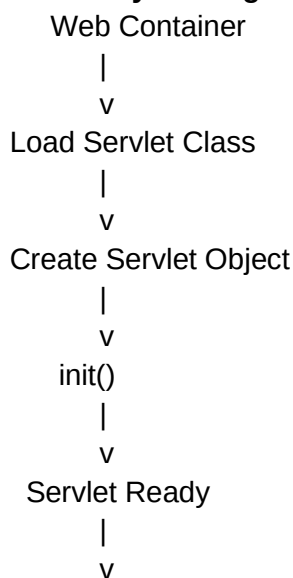
The Servlet Life Cycle describes the sequence of stages through which a servlet passes from its creation to its destruction. The Web Container manages the complete lifecycle of a servlet. The main methods involved are `init()`, `service()`, and `destroy()`.

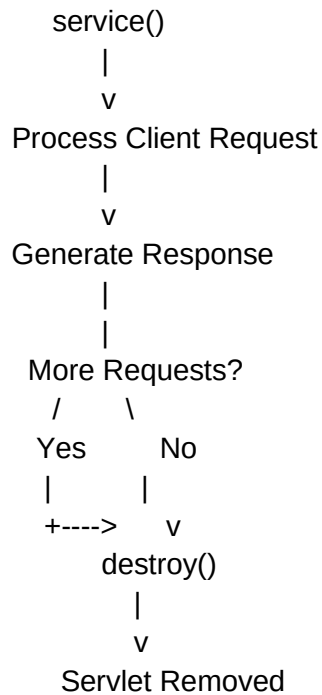
Servlet Life Cycle

The major phases of the Servlet Life Cycle are:

1. Loading and Instantiation
2. Initialization
3. Request Processing
4. Destruction

Servlet Life Cycle Diagram





1. Loading and Instantiation

When the Web Container receives a request for a servlet, it loads the servlet class if it has not already been loaded.

The container then creates an instance of the servlet.

The servlet is normally instantiated only once, and the same servlet instance can handle multiple requests.

2. Initialization – `init()`

After creating the servlet instance, the Web Container calls the `init()` method.

The `init()` method is executed **only once** during the servlet's lifecycle.

It is mainly used for initialization tasks such as:

- Reading configuration information.
- Establishing database connections.
- Initializing resources.
- Preparing required objects.

Example:

```
public void init() {
    System.out.println("Servlet Initialized");
}
```

3. Request Processing – `service()`

After initialization, the servlet is ready to process client requests.

For every request, the Web Container calls the `service()` method.

The `service()` method receives:

- `HttpServletRequest`
- `HttpServletResponse`

Example:

```
public void service(HttpServletRequest request,
    HttpServletResponse response) {
```

```
// Process request
}
```

For an HttpServlet, the service() method determines the appropriate HTTP method and dispatches the request to methods such as:

GET → doGet()

POST → doPost()

The service() method may be called multiple times because a servlet can handle many client requests during its lifetime.

4. Destruction – destroy()

When the Web Container needs to remove the servlet, it calls the destroy() method.

The destroy() method is executed only once.

It is used to release resources such as:

- Database connections.
- Files.
- Network connections.
- Other allocated resources.

Example:

```
public void destroy() {
    System.out.println("Servlet Destroyed");
}
```

Complete Execution Flow

Client Request

|

v

Web Container

|

v

Load Servlet

|

v

Create Servlet Instance

|

v

init() ← Called once

|

v

service() ← Called for each request

|

v

Generate Response

|

v

More Requests

|

+-----> service()

|
v

destroy() ← Called once

Role of the Main Methods

Method	Execution	Purpose
init()	Once	Initializes the servlet
service()	Multiple times	Processes client requests
destroy()	Once	Releases resources before removal

Performance Implications

init()

Since init() is called only once, expensive initialization tasks can be performed here instead of repeating them for every request.

service()

The service() method is executed for every request, so inefficient operations inside it can negatively affect performance. Database queries and other expensive operations should therefore be optimized.

destroy()

The destroy() method ensures that resources are properly released when the servlet is removed. This helps prevent resource leaks.

Conclusion

The Servlet Life Cycle is managed by the Web Container and consists mainly of initialization, request processing, and destruction. The init() method prepares the servlet, service() processes client requests, and destroy() releases resources. Understanding these stages is important for developing efficient and reliable Java web applications.

Question 3: GET vs POST Method Evaluation

Introduction

GET and POST are HTTP methods commonly used for submitting data from a client to a web server. Both methods can be handled by Java Servlets, but they differ in how data is transmitted and in their appropriate use cases.

Comparison of GET and POST

Feature	GET	POST
Purpose	Retrieves data from the server	Sends data to the server
Data Transmission	Data is appended to the URL	Data is sent in the request body
Visibility	Data is visible in the URL	Data is not normally visible in the URL
URL Limitation	Limited by URL length	Can handle larger amounts of data
Bookmarking	Can be bookmarked	Generally not suitable for bookmarking
Caching	Can be cached	Generally not cached in the same way
Security	Not suitable for sensitive data	Better suited for sensitive data, but HTTPS is still required

Feature	GET	POST
Common Use	Search, filtering, retrieving information	Login, registration, form submission

GET Method

The GET method is mainly used to request or retrieve information from a server.

Example

```
<form action="SearchServlet" method="get">
  <input type="text" name="query">
  <input type="submit" value="Search">
</form>
```

The submitted data is included in the URL.

For example:

SearchServlet?query=java

Advantages of GET

1. Simple to use.
2. Parameters can be bookmarked.
3. Useful for retrieving information.
4. Suitable for search and filtering operations.
5. Can be cached by browsers and intermediaries.

Limitations of GET

1. Data is visible in the URL.
2. URL length limits the amount of data that can be transmitted.
3. Sensitive information such as passwords should not be sent using GET.
4. URLs may be stored in browser history or logs.

POST Method

The POST method is mainly used to send data to the server for processing.

Example

```
<form action="LoginServlet" method="post">
  <input type="text" name="username">
  <input type="password" name="password">
  <input type="submit" value="Login">
</form>
```

The form data is sent in the HTTP request body rather than being appended to the URL.

Advantages of POST

1. Data is not displayed in the URL.
2. Suitable for submitting larger amounts of data.
3. Appropriate for form submissions.
4. Suitable for operations that modify server-side data.
5. Better suited than GET for sensitive form data when combined with HTTPS.

Limitations of POST

1. Data cannot normally be bookmarked in the same way as GET parameters.
2. Requests generally cannot be cached in the same way as GET.
3. HTTPS is still necessary when transmitting sensitive information.

Security Considerations

GET should not be used for sensitive information because parameters appear in the URL.
For example:

login?username=student&password=12345

This information may appear in browser history, server logs, or other locations.

POST is more appropriate for sensitive form submissions because the data is placed in the request body. However, **POST alone does not provide encryption**. HTTPS should be used to protect sensitive data during transmission.

Appropriate Scenarios

GET is Appropriate For

- Search operations.
- Retrieving records.
- Filtering products.
- Viewing web pages.
- Querying non-sensitive information.

POST is Appropriate For

- Login forms.
- Registration forms.
- Uploading files.
- Submitting large amounts of data.
- Creating or modifying server-side resources.

Servlet Implementation

Handling GET

```
protected void doGet(HttpServletRequest request,
    HttpServletResponse response) {

    String name = request.getParameter("name");

    // Process GET request
}
```

Handling POST

```
protected void doPost(HttpServletRequest request,
    HttpServletResponse response) {

    String username = request.getParameter("username");

    // Process POST request
}
```

Conclusion

GET is mainly suitable for retrieving non-sensitive information and operations where the parameters can safely appear in the URL. POST is more appropriate for submitting data, especially larger or sensitive form data. However, security depends on using HTTPS, proper validation, authentication, and other appropriate security mechanisms.

Question 4: Session Tracking Techniques

Introduction

HTTP is a stateless protocol, which means that the server does not automatically remember previous requests from the same client. Session tracking techniques are used to maintain user-specific information across multiple requests in a web application.

The three session tracking mechanisms considered here are:

1. HttpSession
2. Cookies
3. URL Rewriting

1. HttpSession

HttpSession is a server-side session tracking mechanism provided by Java Servlets. The server creates a session for a user and stores session-related information on the server.

Example

```
HttpSession session = request.getSession();
```

```
session.setAttribute("username", "Student");
```

```
String name = (String) session.getAttribute("username");
```

Advantages

- Stores session data on the server.
- Convenient for maintaining user information.
- Suitable for login and shopping-cart applications.
- Can store multiple attributes.

Limitations

- Uses server memory.
- Large numbers of active sessions can increase server resource usage.
- Session management is required when multiple servers are used.

2. Cookies

A cookie is a small piece of information stored on the user's browser. The browser sends the cookie back to the server with subsequent requests.

Example

```
Cookie cookie = new Cookie("username", "Student");
```

```
response.addCookie(cookie);
```

To retrieve a cookie:

```
Cookie[] cookies = request.getCookies();
```

```
if (cookies != null) {  
    for (Cookie cookie : cookies) {  
        if (cookie.getName().equals("username")) {  
            System.out.println(cookie.getValue());  
        }  
    }  
}
```

Advantages

- Easy to implement.

- Stored on the client side.
- Reduces server-side storage requirements.
- Can persist across requests and, depending on configuration, browser sessions.

Limitations

- Users can disable cookies.
- Cookies are stored on the client.
- Sensitive information should not be stored directly in cookies.
- Cookies have size limitations.

3. URL Rewriting

URL rewriting adds session information to the URL when cookies are unavailable.

Example

```
String url = response.encodeURL("profile");
```

The session identifier can be included in the generated URL.

Example:

```
profile;jsessionid=ABC123
```

Advantages

- Works when cookies are disabled.
- Does not depend on browser cookie support.
- Can be used as an alternative session tracking mechanism.

Limitations

- Session information may appear in the URL.
- URLs can become longer.
- URLs containing session identifiers may create security risks.
- Less convenient and less clean than cookie-based tracking.

Comparison

Feature	HttpSession	Cookies	URL Rewriting
Storage	Server	Client	URL
Server Resource Usage	Higher	Lower	Lower
Cookie Required	Usually uses session cookie for ID	Yes	No
Security	Relatively better for storing data server-side	Depends on configuration	Session ID exposed in URL
Persistence	Session-based	Can be persistent	Depends on session
Ease of Use	Easy	Easy	Moderate
Best Use	Login and user sessions	Preferences and identifiers	When cookies are disabled

Security Concerns

HttpSession

Session identifiers should be protected against session hijacking. HTTPS should be used to protect session information during transmission.

Cookies

Security attributes such as Secure and HttpOnly can help protect cookies. Sensitive information

should not be directly stored in client-side cookies.

URL Rewriting

URL rewriting can expose session identifiers in browser history, bookmarks, logs, or shared links. Therefore, it should be used carefully.

Use-Case Comparison

HttpSession

Suitable for:

- User login sessions.
- Shopping carts.
- User-specific application data.

Cookies

Suitable for:

- User preferences.
- Remembering non-sensitive settings.
- Session identifiers.

URL Rewriting

Suitable for:

- Applications where cookies are disabled.
- Maintaining sessions as an alternative to cookies.

Conclusion

HttpSession, Cookies, and URL Rewriting provide different approaches to session tracking.

HttpSession is suitable for server-side user sessions, Cookies are useful for storing small client-side information, and URL Rewriting can maintain sessions when cookies are unavailable. The appropriate mechanism depends on application requirements, security considerations, and persistence needs.

Question 5: Cookies vs Sessions – Critical Analysis

Introduction

Cookies and Sessions are commonly used in web applications to maintain user-related information across multiple HTTP requests. Since HTTP is stateless, these mechanisms help applications recognize users and maintain their state during interaction with the website.

Comparison of Cookies and Sessions

Feature	Cookies	Sessions
Storage Location	Client-side	Server-side
Data Storage	Small amount of data	Can store more application-specific data
Security	More exposed because data is stored on the client	Generally safer for sensitive session data because data is stored on the server
Server Memory	Low usage	Uses server resources
Persistence	Can be persistent depending on expiry settings	Usually associated with a user session
Performance	Reduces server-side storage	Requires server-side session management

Feature	Cookies	Sessions
	requirements	
User Control	Users can delete or disable cookies	Managed by the server
Common Usage	Preferences, identifiers, and session IDs	Login sessions, shopping carts, user-specific data

Cookies

Cookies are small pieces of data stored in the user's browser. The browser sends the cookie to the server with subsequent requests.

Advantages

1. Easy to implement.
2. Reduces the need for server-side storage.
3. Can persist between browser sessions when configured with an expiry time.
4. Useful for storing preferences and identifiers.

Disadvantages

1. Users can disable or delete cookies.
2. Cookies are stored on the client side.
3. Cookies have size limitations.
4. Improperly protected cookies can be exposed to security attacks.

Sessions

Sessions store user-specific information on the server. A session identifier is generally associated with the client so that the server can identify the user's session.

Advantages

1. Session data is maintained on the server.
2. Suitable for sensitive application state.
3. Useful for maintaining login information.
4. Can store multiple attributes related to a user.

Disadvantages

1. Requires server memory and resources.
2. A large number of active sessions can increase server load.
3. Session management becomes more complex in distributed applications.
4. Session identifiers must be protected against attacks such as session hijacking.

Security Risks

Cookie Security

Cookies containing session identifiers should be protected using appropriate security settings.

The Secure attribute ensures that cookies are sent over HTTPS, while the HttpOnly attribute helps prevent client-side scripts from accessing the cookie.

Sensitive information should not be stored directly in cookies.

Session Security

Sessions should use secure session identifiers and HTTPS to reduce the risk of session hijacking. Sessions should also be invalidated when a user logs out.

Performance Considerations

Cookies reduce server-side storage requirements because the data is stored on the client. However, cookies are transmitted with requests, which can increase request size.

Sessions require server-side resources because session information must be stored and managed by the server. With a large number of users, session storage can increase server memory requirements.

Real-Time Use Cases

E-Commerce Website

A shopping cart can use a session to maintain products selected by a logged-in user.

User → Login



Session Created



Products Added to Cart



Session Maintains Cart Information

Online Banking

Sessions can be used to maintain authenticated user information after login. Secure session management is important because financial information is highly sensitive.

Social Media

Cookies can be used to maintain identifiers and user preferences, while sessions can maintain authenticated user state.

Online Shopping Preferences

Cookies can store non-sensitive preferences such as language, theme, or selected settings.

Critical Analysis

Cookies and sessions are not necessarily competing technologies. They are often used together.

For example, a web application may store a **session identifier in a cookie**, while the actual user session data is stored on the server.

This approach combines the convenience of cookies with the security and control of server-side sessions.

Conclusion

Cookies are suitable for storing small amounts of client-side information such as preferences and identifiers, while Sessions are more appropriate for maintaining server-side user state such as authentication and shopping-cart information. For secure web applications, a combination of cookies and server-side sessions is often the most practical approach, with appropriate security measures such as HTTPS, Secure, and HttpOnly attributes.